PAN-ATLANTIC UNIVERSITY

Decision Making and Loops

CSC 101 Practical · Week 4

2026/2027 Session

Pan-Atlantic University

Today's Practical

Part 1

Reading user input

Your programs finally talk back. std::io, read_line and parse. Practices 1 and 2.

Part 2

Decision making

if, if else, the else if ladder and match. Practices 3 to 5.

Part 3

Loops

for, while and loop, plus break and continue. Practices 6 to 9.

Projects

Two class projects

Quadratic roots and an employee incentive calculator.

PART 1 OF 3

Reading User Input

std::io lets your program ask a question and use the answer.



How to Read User Input

1. **Bring the input tools in**

```
use std::io;
```

2. **Make a mutable String to receive the text**

```
let mut name = String::new();
```

3. **Ask the keyboard**

```
io::stdin().read_line(&mut name)
```

4. **Handle the risk**

```
.expect("Failed to read input");
```

Reading can fail, so Rust makes you say what happens if it does.

5. **Numbers need one more step**

```
age.trim().parse()
```

read_line gives text. trim() drops the newline, parse() converts.

Practice 1: Name and Age

A tiny student information system: the program asks for your name and age, then repeats them back.

```rust
// Rust program to output name and age

use std::io;

fn main() {
    println!("\nStudent Information Management System!");

    // input name
    println!("\nPlease Enter your name.");
    let mut name = String::new();
        io::stdin()
        .read_line(&mut name)
        .expect("Failed to read input");
    println!("Your name is: {}", name);

    // input age
    println!("\nEnter your age.");
    let mut age = String::new();
        io::stdin().read_line(&mut age).expect("Failed to read input");
    let age:i32 = age.trim().parse().expect("Input not an integer");
    println!("Your age is: {}", age);
}
```

1. **Two questions, two reads**
   Name stays a String. Age becomes u32 through trim and parse.

2. **Text just gets printed**
   `println!("Your name is: {}", name);`

3. **Numbers get converted first**
   `.parse().expect("Input not an integer")`

Practice 2: Triangle From Three Sides

Same pattern, more maths: the area of a triangle from its three sides.

```rust
// Rust program to calculate the area of a triangle given three sides

use std::io;

fn main()
{
    let mut input1 = String::new();
    let mut input2 = String::new();
    let mut input3 = String::new();

    println!("Enter first edge of triangle: ");
    io::stdin().read_line(&mut input1).expect("Not a valid string");
    let a:f32 = input1.trim().parse().expect("Not a valid number");

    println!("Enter second edge of triangle: ");
    io::stdin().read_line(&mut input2).expect("Not a valid string");
    let b:f32 = input2.trim().parse().expect("Not a valid number");

    println!("Enter third edge of triangle: ");
    io::stdin().read_line(&mut input3).expect("Not a valid string");
    let c:f32 = input3.trim().parse().expect("Not a valid number");

    let s:f32 = (a + b + c) / 2.0;
    let mut area:f32 = s * (s - a) * (s - b) * (s - c);
    area = area.sqrt();

    println!("Area of a triangle: {}", area);
}
```

1. **Three sides from the keyboard**
   Each read_line arrives as text; parse turns it into f32.

2. **Half the perimeter first**
   `let s: f32 = (a + b + c) / 2.0;`

3. **Heron's formula, then the square root**
   `area.sqrt()`

PART 2 OF 3

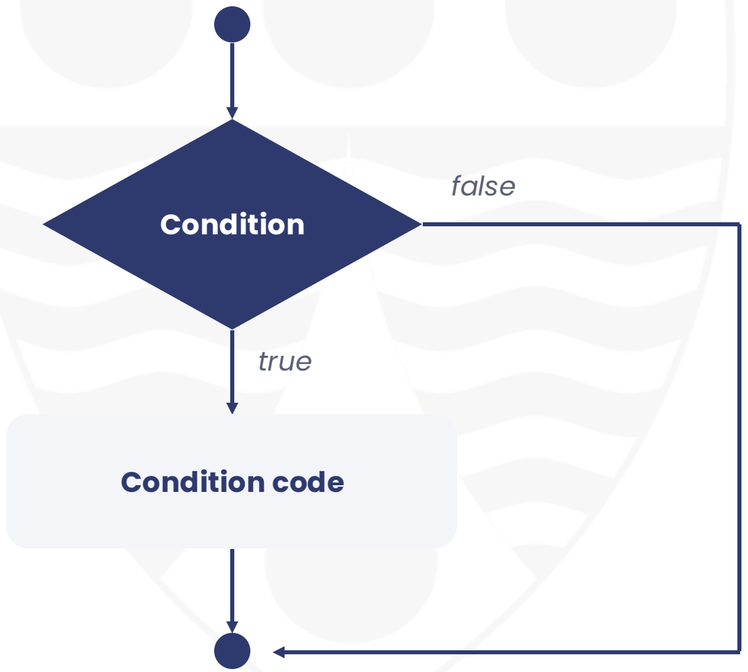
Decision Making

Programs that choose a path: if, if else, the else if ladder and match.



How a Decision Flows

Decision making structures test a condition. If it holds, one block of code runs; if not, the program simply moves on.



Four Ways to Decide

`if`	One Boolean expression, one block of statements. Runs only when the expression is true.
`if else`	Adds a second path: the else block runs when the expression is false.
`else if` and nested `if`	Chain more conditions, or put an if inside another, to test several cases in turn.
`match`	Tests one variable against a list of values. Rust's heavyweight decision tool, covered fully in later weeks.

The if Statement

A condition guards a block

The Boolean expression is evaluated first. True, and the block inside the braces runs.

False? Skip it

Execution jumps straight past the closing brace to whatever comes next.

```
if boolean_expression {
    // runs only when the expression is true
}
```

Practice 3: if in Action

Triangle area again, from base and height this time, with the calculation guarded by an if.

```rust
// Rust program to calculate the area of a
// triangle for a given base and height

use std::io;

fn main()
{
    let mut input1 = String::new();
    let mut input2 = String::new();

    println!("Enter base: ");
    io::stdin().read_line(&mut input1).expect("Not a valid string");
    let base:f32 = input1.trim().parse().expect("Not a valid number");

    println!("Enter height: ");
    io::stdin().read_line(&mut input2).expect("Not a valid string");
    let height:f32 = input2.trim().parse().expect("Not a valid number");

    if base > 0.0 {
        let area:f32 =(base * height) / 2.0;
        println!("Area of a triangle: {}", area);
    }
}
```

1. **A guard before the maths**
   `if base > 0.0`

2. **Area only when it passes**
   `(base * height) / 2.0`

3. **No else block here**
   If base is zero or negative, this program prints nothing at all.

The if else Statement

Two paths, one taken

The if block runs when the expression is true. The optional else block runs when it is false.

Never both

Exactly one of the two blocks executes, every time.

```
if boolean_expression {
    // runs when true
} else {
    // runs when false
}
```

Practice 4: The Party Gate

Read a name and an age, then let if else decide who gets in.

```rust
// Rust program to determine age pass

use std::io;

fn main() {

    let mut input1 = String::new();
    let mut input2 = String::new();

    println!("Enter your name: ");
    io::stdin().read_line(&mut input1).expect("Not a valid string");

    println!("Enter your age: ");
    io::stdin().read_line(&mut input2).expect("Not a valid string");
    let age:i32 = input2.trim().parse().expect("Not a valid number");

    if age >= 18 {
      println!("Welcome to the party {}!", input1);
   } else {
      println!("Oops, you are not of age to enter the party {}", input1);
   }
}
```

1. **One condition decides**
   `if age >= 18`

2. **True gets a welcome**
   Welcome to the party, plus the name.

3. **False gets the else block**
   Oops, you are not of age to enter the party.

The else if Ladder

For more than two cases

Chain conditions and Rust tests them from the top.

Three rules

At most one else, and it comes last. Any number of else ifs, all before the else. Once one branch succeeds, the rest are never tested.

```
if expression1 {
    // expression1 is true
} else if expression2 {
    // expression2 is true
} else {
    // both were false
}
```

Practice 5: The Height Ladder

Read a height in centimetres and place it in a category with an else if ladder.

```rust
// Rust program to read the height of a person
// and then print if person is tall, dwarf,
// or average height person

use std::io;

fn main()
{
    let mut input = String::new();

    println!("\nEnter Your Height (in centimetres):");
    io::stdin().read_line(&mut input).expect("Not a valid string");
    let height:f32 = input.trim().parse().expect("Not a valid number");

    if height >= 150.0 && height <= 170.0
    {
        println!("You are of average height person");
    }
    else if height > 170.0 && height <= 195.0
    {
        println!("You are tall");
    }
    else if height < 150.0 && height > 100.0
    {
        println!("You are dwarf");
    }
    else
    {
        println!("Abnormal height");
    }
}
```

1. **Ranges tested in order**
   150 to 170 average, 170 to 195 tall, then short, then abnormal.

2. **Two tests joined with &&**
   `height >= 150.0 && height <= 170.0`

3. **First match wins**
   As soon as one branch runs, the remaining branches are skipped.

PART 3 OF 3

Loops

Doing things many times: for, while and loop, with break and continue.



How a Loop Flows

A loop runs a block of statements again and again while a condition holds. When the condition fails, the program moves on.



Two Kinds of Loops

Definite

`for`

The number of repetitions is fixed and known: run this block ten times, or once per item in a set of values.

Indefinite

`while` · `loop`

The number of repetitions is unknown in advance. while repeats as long as its condition holds; loop is a while(true) that runs until you break out.

The for Loop

A definite loop

Executes the block a specified number of times, or once for each value in a fixed set.

Reads like English

For every value in this range, do the following.

```
for temp_variable in lower_bound..upper_bound {
    // statements
}
```

Mind the two dots: the upper bound itself is not included in the range.

Practice 6: Counting With for

A counter where the user chooses where it starts and where it stops.

```rust
// Rust program to count numbers

use std::io;

fn main(){

    println!("Enter lower bound");
    let mut input1 = String::new();
    io::stdin().read_line(&mut input1).expect("Failed to read input");
    let lower_bound:i32 = input1.trim().parse().expect("Failed to input");

    println!("Enter upper bound");
    let mut input2 = String::new();
    io::stdin().read_line(&mut input2).expect("Failed to read input");
    let upper_bound:i32 = input2.trim().parse().expect("Failed to input");

    for x in lower_bound..upper_bound{ // upper_bound is not inclusive

      println!("Count Level is {}",x);
   }
}
```

1. **Bounds come from the keyboard**
   Two reads, both parsed to i32.

2. **The loop walks the range**
   `for x in lower_bound..upper_bound`

3. **Upper bound is not inclusive**
   Bounds 1 and 5 count 1, 2, 3, 4.

Practice 7: The while Loop

An indefinite loop: while repeats its block for as long as the condition stays true.

```rust
use std::io;

fn main() {

    println!("Enter a number");
    let mut input1 = String::new();
    io::stdin().read_line(&mut input1).expect("Failed to read input");
    let mut num:i32 = input1.trim().parse().expect("Failed to input");

    while num < 10 {

        println!("inside loop number value is {}",num);
        num+=1;
    }
    println!("outside loop number value is {}",num);
}
```

1. **The test comes first**
   `while num < 10`

2. **Something must move**
   `num += 1;`
   Forget this line and the condition never fails: an infinite loop.

3. **Life after the loop**
   The final print shows num is 10 once the loop lets go.

Practice 8: loop and break

loop runs forever on its own. break is how you take control back.

```rust
fn main(){

    //while true

    let mut x = 0;
    loop {
        x+=1;
        println!("x={}",x);

        if x==15 {
            break;
        }
    }
}
```

1. **No condition at all**
   `loop {`
2. **So you provide the exit**
   `if x == 15 { break; }`
3. **break leaves the loop**
   Control jumps to the first line after the loop's closing brace.

Practice 9: continue

continue skips the rest of the current iteration. Unlike break, the loop itself keeps going.

```rust
fn main() {

    let mut count = 0;

    for num in 1..21 {
        if num > 10 {
          println!("{:?}",num);
            continue;
        }
        count+=1;
    }
    println! (" The count of values greater than 10 (between 1 and 20) is: {} ",count);
    //outputs 10
}
```

1. **Skip, don't exit**
   `if num > 10 { continue; }`

2. **The iteration ends early**
   continue jumps back to the top of the loop for the next value.

3. **So the count stops at 10**
   count only rises for 1 to 10; the values above are skipped past it.

CLASS PROJECTS

Projects I and II

Quadratic roots and employee incentives. Both need input, decisions and care.



Project I: Quadratic Roots

Given the values of a, b and c, find the roots of a quadratic equation with a Rust program. Read all three values from the keyboard.

Hint: the discriminant decides everything

```
d = b*b - 4.0*a*c
```

d greater than zero: two distinct roots

d equal to zero: exactly one real root

d less than zero: no real roots

Project II: The Incentive Calculator

Take an employee's experience and age as input, and determine the annual incentive from these criteria:

Experienced, age 40 or more	₦1,560,000
Experienced, age 30 to 39	₦1,480,000
Experienced, below 28	₦1,300,000
Not experienced	₦100,000

PAN-ATLANTIC UNIVERSITY

Questions?

Homework: Practices 1 to 9 and both projects, pushed to week-4.